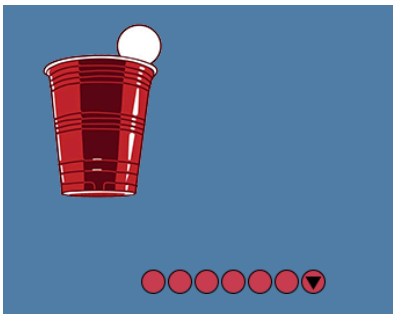
P05 Team Party Hopping

Overview

Our Agents are back on the party circuit, but now they have colors and they're moving in teams!



Grading Rubric

5 points	**Pre-assignment Quiz**: accessible through Canvas until 10:00 PM on **2/28**.
20 points	**Immediate Automated Tests**: accessible by submission to Gradescope. You will receive feedback from these tests *before* the submission deadline and may make changes to your code in order to pass these tests. Passing all immediate automated tests does **not** guarantee full credit for the assignment.
15 points	**Additional Automated Tests**: these will also run on submission to Gradescope, but you will not receive feedback from these tests until after the submission deadline.
10 points	**Manual Grading Feedback**: your code will be manually reviewed for commenting, style, and algorithms/readability.
50 points	**MAXIMUM TOTAL SCORE**

Learning Objectives

After completing this assignment, you should be able to:

- **Describe** the object-oriented principle of polymorphism and how it affects program design and execution.
- **Explain** the function of the `@Override` tag and the purpose of overriding a method.
- **Implement** a data type in Java with a parent class other than Object and/or which explicitly incorporates an interface, and **explain** the utility of these constructs.

Additional Assignment Requirements and Notes

Keep in mind:

- Pair programming is **ALLOWED** for this assignment; however, you must have ***already registered*** with a partner by the time this writeup was released. If you have not registered a partner for P05, you ***must*** complete the assignment individually.

- The ONLY external libraries you may use in your program are:
  ```
  java.io.File
  processing.core.{PImage, PApplet}
  java.util.{ArrayList, Random}
  ```
  Use of any other packages (outside of java.lang) is NOT permitted.

- **NOTE:** The automated tests in Gradescope ***do not have access to the full Processing library***. If you use any methods in your program outside of those mentioned in the starter code or this writeup, your code may work on your local machine but **FAIL** the automated tests. This program can be completed successfully using ONLY these methods.

- You are allowed to define any **local** variables you may need to implement the methods in this specification (inside methods). You are NOT allowed to define any additional instance or static variables or constants beyond those specified in the write-up.

- All methods must be static. You are allowed to define additional **private** helper methods.

- All classes and methods must have their own Javadoc-style method header comments in accordance with the CS 300 Course Style Guide.

- Any source code provided in this specification may be included verbatim in your program without attribution.

- **All other sources must be cited explicitly in your program comments**, in accordance with the Appropriate Academic Conduct guidelines. As in P04, for full credit all students MUST cite AT LEAST ONE source.

- Any use of ChatGPT or other large language models ***must be cited*** AND your submission MUST include **screenshots** of your interactions with the tool ***clearly showing all prompts and responses in full***. Failure to cite or include your logs is considered academic misconduct and will be handled accordingly.

- **Run your program locally before you submit to Gradescope**. If it doesn't work on your computer, *it will not work on Gradescope*.

Need More Help?

Check out the resources available to CS 300 students here.

CS 300 Assignment Requirements

You are responsible for following the requirements listed on both of these pages on all CS 300 assignments, whether you've read them recently or not. Take a moment to review them if it's been a while:

- Appropriate Academic Conduct, which addresses such questions as:
  - How much can you talk to your classmates?
  - How much can you look up on the internet?
  - How do I cite my sources?
  - and more!

- Course Style Guide, which addresses such questions as:
  - What should my source code look like?
  - How much should I comment?
  - and more!

1. Getting Started

1. Create a new project in Eclipse, called something like **P05 Team Party Hopping**.
   a. Ensure this project uses Java 17. Select "JavaSE-17" under "Use an execution environment JRE" in the New Java Project dialog box.
   b. Do **not** create a project-specific package; use the default package.

2. Download two (2) Java source files from the assignment page on Canvas:
   a. **TeamTester.java** (includes a main method)
   b. **TeamManagementSystem.java** (GUI program – includes a main method)

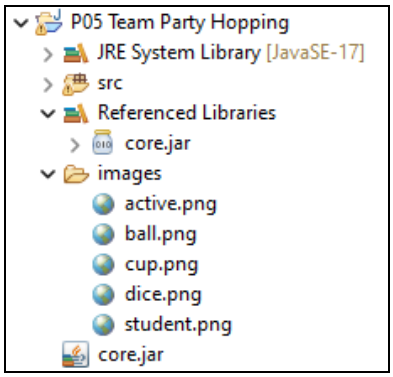
3. Download two (2) more starter files from the assignment page on Canvas:
   a. **core.jar** (the Processing library JAR file, exactly as distributed)
   b. **images.zip** (exactly the same as P02, you can reuse that one if you prefer)

4. Create five (5) Java source files within your projects src folder, ***none*** of which include a main method:
   a. **Clickable.java** (interface!)
   b. **Agent.java** (implements the Clickable interface)
   c. **Lead.java** (child class of Agent)
   d. **Party.java** (implements the Clickable interface)
   e. **Team.java**

To complete setup, add **core.jar** to your project as in P02 and add it to your build path ("Build Path" → "Add to Build Path…").

Unzip the **images.zip** file and add the images folder to your project as in P02.

If you encounter any issues, **STOP** and fix them now before you continue.



1.1 The Clickable interface

To begin, **create the Clickable interface** in accordance with its Javadocs. This should be very quick.

Once you've finished writing the interface, modify your Agent and Party classes so that they *implement* the interface without errors – stub methods are fine to begin with.

2. Instantiable Class Implementation Suggestions

- Begin by implementing the instantiable classes described in the Javadocs, and test them thoroughly before you begin writing the GUI program. The GUI may help you uncover additional bugs later, but if you have completed (and debugged) Agent, Lead, Team, and Party before you start on the TeamManagementSystem, things will be much easier for you.
- **Read the Javadocs** (linked on the assignment page) **carefully**. Almost everything you need to know is contained in them.

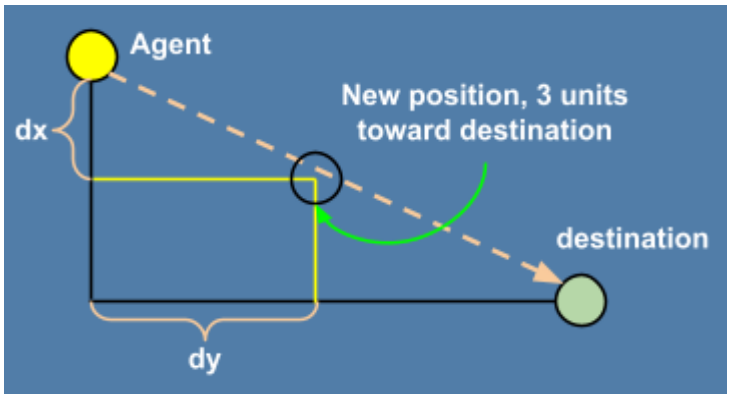
2.1 Agent Implementation

The Agent class (and all classes you will write from scratch) is primarily described in the Javadocs linked on Canvas, but here are a few extra insights:

This is your base class; instead of a 😊 this time we're looking at a 🟡 (or a different colored circle, depending on whether the agent is currently active or a member of a Team).

- To draw an agent, first set PApplet's `fill()` to the correct color; this color is given by your `getColor()` method. Then use the `circle()` method with your agent's (x,y) coordinates and diameter to get the circle to appear correctly.
- When implementing the **dragging** behavior, think about these steps:
  - Calculate how much the mouse has moved between its current (x,y) coordinates and the previous (x,y) coordinates. In math terms, we'd call this the mouse's dx and dy.
  - Update your agent's position by that same dx and dy.
  - Store the current location of the mouse as the previous coordinates, for next time.

- When implementing the **moving** behavior, you will be updating your Agent's location to be a total of 3 units closer to the destination along what is most likely the hypotenuse of a triangle:



2.2 Lead Implementation

The Lead class extends Agent and retains most of its behavior, but introduces some modifications.. Clicking on a Lead activates its entire team, for instance, and so we need to be able to SEE which Agents are actually Leads:

- To draw a team lead, first draw it as you would a normal Agent. Then, set the `fill()` to black (easiest way to do that is just use 0 as the parameter) and use the `triangle()` method to draw an inverted triangle with the following three vertices relative to the team lead's (x,y) position:
  - (x-diameter/3, y-diameter/5)
  - (x+diameter/3, y-diameter/5)
  - (x, y+diameter/3)

✅ **CHECKPOINT 1: AGENT TESTS**
You should now be able to write (and pass!) the first three tests in TeamTester.

2.3 Party Implementation

A Party in this program works almost the same as it did in P02, except now to send Agents to them you'll be clicking on them instead of pressing keyboard keys. That means they need the Clickable interface implemented:

- When the mouse is *pressed* on a Party, nothing happens. You can leave this method entirely blank (or even better, put a comment in the method that says something like "this method is intentionally left empty" so you know later that you didn't forget to write it).

- When the mouse is released on a Party, that's when you'll need to see if there's an active Team. You may want to add a stub method to Team for the `sendToParty()` method so you can finish the behavior of this method here.

- The `isMouseOver()` and `draw()` methods should be very familiar; you'll want the `width` and `height` properties of the PImage you're using to help figure out whether the mouse is over it, and the `image()` method from your TeamManagementSystem's parent class PApplet to draw that image to the application window.

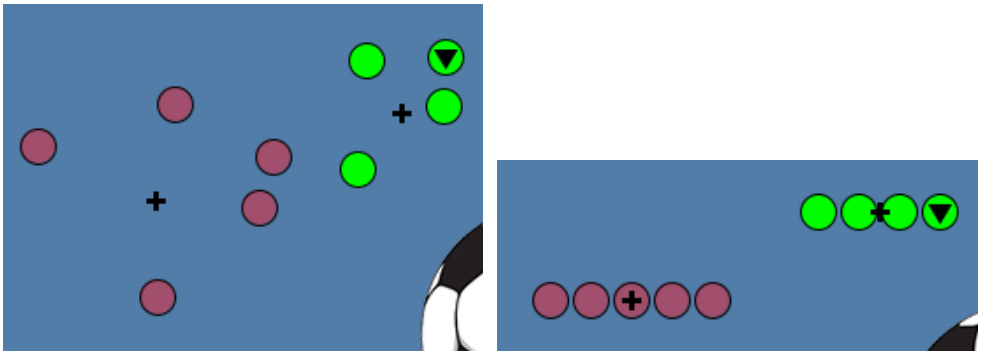
2.4 Team Implementation

This is the new piece: a grouping class for multiple Agents. While this is largely a management class for a single ArrayList, there are a couple of application-specific pieces:

- Teams can have 0 or 1 Lead members. Attempts to add any more than that should cause errors. The `instanceof` operator is your friend here!

- The centerX/centerY coordinates we're asking you to calculate are not weighted in any way, they're just halfway between the leftmost and rightmost Agents (or topmost/bottommost Agents).
    - A Team of **one Agent** will have a centerX/centerY *exactly equal* to that Agent's position.
    - A **lined-up Team** will have a centerY exactly equal to ALL its members' y-coordinate, and the centerX will either be the x-coordinate of the center member (if there are an odd number of members) or exactly between the two centermost members (if there are an even number).

    - In these screenshots, the centerX/centerY coordinates are marked with + before and after lining up two different teams:

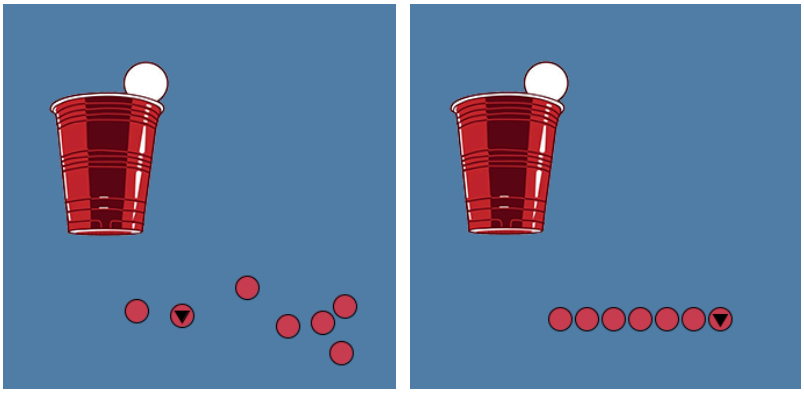


If you would like to include this visualization **for debugging purposes**, you can add the following code to the end of your TeamManagementSystem's `draw()` method:

```
// draw the centerX/centerY of the team as a +
float centerX = t.getCenterX();
float centerY = t.getCenterY();

fill(0);
rect(centerX-1,centerY-5,2,10);
rect(centerX-5,centerY-1,10,2);
```

- Sending all of the members to a Party should *preserve the current agent formation* - don't just send all of the agents to one (x,y) position! (See the sample video.)
- Having the members line up will take a little math. The specific implementation of this behavior *may* best be done once you've got the GUI working so you can debug it, but ultimately this method should take you from a scattered collection of Agents (left) to a nice, neat, evenly-spaced line with **3 pixels between each Agent** (right):



(See the Team Javadocs for more details on how lining up should work.)

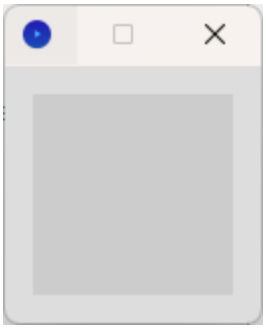
✅ **CHECKPOINT 2: TEAM TESTS**
You should now be able to complete (and pass!) all tests in your TeamTester.

3. The TeamManagementSystem GUI Program

The steps for completing this class are largely contained in numbered TODOs in the provided starter file, but we have a few additional hints for you here.

3.1 Define the `settings()` and `setup()` callback methods

If you just run the TeamManagementSystem program as it stands, you should see a weird, tiny window:



In the `settings()` method (which was previously handled by the provided Utility class), add a call to `size()` with a width of 800 and a height of 600 to make it a more recognizable size.

For TODO #2 in the `setup()` method, you'll need to give those instantiable classes you wrote a reference to this class. Notice that the TeamManagementSystem extends a class called processing.core.PApplet – this is the main Processing class that handles the runtime loop and all of the callback methods! It's super complicated! But we're inheriting from it, so we get all of its functionality for free and we'll never actually need to think about it. Object-oriented programming!!

- Both Agent and Party will need references to the ***CURRENT*** TeamManagementSystem object. Don't call a constructor here!! You've already got a reference to a TeamManagementSystem object – you're in an instance method.

To make the Party setup look like it did back in P02, you can use the following locations. You're not required to use these exact places, but you ARE required to make exactly one (1) cup, one (1) dice, and one (1) ball Party for your submitted program. (What you do on your own time is your business.)

- **Cup**: x = 200, y = 125
- **Dice**: x = 600, y = 150
- **Ball**: x = 400, y = 450

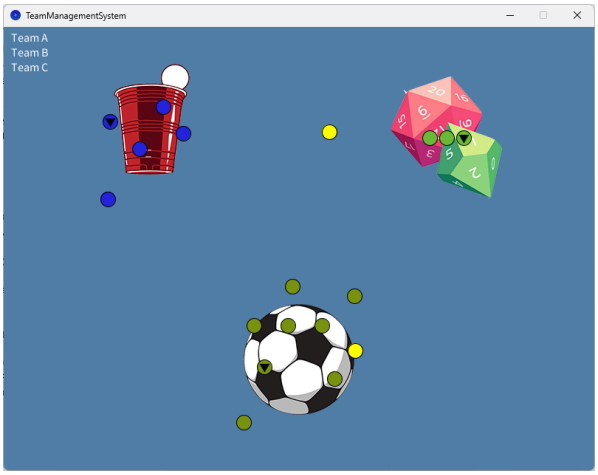
All other TODOs here should be relatively self-explanatory, but remember the `color()` method for creating color-representative int values.

3.2 Define `draw()` and its helper methods

The main new thing here is the selection box – see the video on the assignment page for what this should look like.

3.3 Define other callback methods and helpers

The rest of the callback methods (mousePressed, mouseReleased, and keyPressed) will look largely familiar, but have new behavior. Follow the TODOs carefully, and run your program frequently to verify that it's working the way you expect it to.



Assignment Submission

Hooray, you've finished this CS 300 programming assignment!

Once you're satisfied with your work, both in terms of adherence to this specification and the academic conduct and style guide requirements, make a final submission of your source code to Gradescope.

For full credit, please submit the following files (**source code**, *not* .class files):

- **Agent.java**
- **Clickable.java**
- **Lead.java**
- **Party.java**
- **Team.java**
- **TeamManagementSystem.java**
- **TeamTester.java**

Additionally, if you used generative AI at any point during your development, ***you must include screenshots*** or a text-based log showing your FULL interaction with the tool(s).

Your score for this assignment will be based on the submission marked "**active**" prior to the deadline. You may select which submission to mark active at any time, but by default this will be your most recent submission.

Copyright notice

Additional Feature Suggestions

Once you've submitted your code, you can keep adding new things if you like (and if you do – please show us! That's rad and we want to see, but it'll mess up the autograder so make sure you've submitted a writeup-compliant version first.)

Suggestions for additional features, to get you started:

- These agents don't hover like they did in P02. Add that functionality back in!
- You can only use the "click on a Party to send Agents there" functionality for a whole team; add functionality for sending all active agents to a party.
- Team's `addMember()` method is only used for testing purposes – add something to your GUI program to add a new member to an existing team.
- BE CREATIVE!!